ADMINISTRATION SYSTÈME

Gatus

Supervision de Services Web — Installation sous Debian 12

Auteur	**bthiery (Kalandy)**
Système	**Debian 12**
Service	**Gatus**
Port	**8080**
Accès	**http://192.168.1.84:8080**

Partie I - Objectif

Cette documentation décrit l'installation, la configuration et la mise en service de Gatus sur Debian 12. Gatus est un outil open-source permettant de superviser des sites web, des API et des services réseau. Il offre une interface web ainsi que l'envoi d'alertes automatiques en cas de défaillance.

Partie II - Installation de Golang

Gatus est développé en langage Go. Avant de le compiler, il est nécessaire d'installer et de configurer l'environnement Golang.

2.1 - Mettre à jour le système

```
sudo apt update && sudo apt upgrade -y
```

2.2 - Installer puis remplacer Golang

➢ Action : Installer la version des dépôts Debian, puis la remplacer par la version officielle plus récente.

```
sudo apt install golang-go -y
sudo apt remove golang-go -y
```

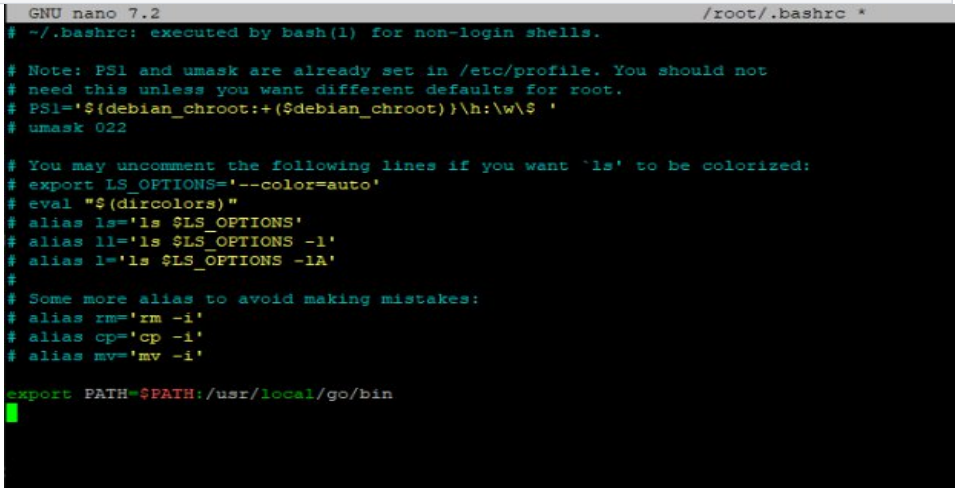
2.3 - Télécharger et installer la version officielle

```
cd /tmp
wget https://go.dev/dl/go1.24.4.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.24.4.linux-amd64.tar.gz
```

2.4 - Ajouter Go au PATH du système

➢ Action : Ouvrir le fichier .bashrc et ajouter la ligne d'export : nano ~/.bashrc

```
export PATH=$PATH:/usr/local/go/bin
source ~/.bashrc
```



2.5 - Vérifier la version installée

```
go version
```

```
go version go1.24.7 linux/amd64
```

✓ **Résultat :** *La version de Go installée s'affiche dans le terminal.*

Partie III - Téléchargement et Compilation de Gatus

3.1 - Installer Git et cloner le dépôt

```
sudo apt install git -y
git clone https://github.com/TwiN/gatus.git
cd gatus
```

3.2 - Compiler Gatus

➢ **Action : La commande go build compile le code source et crée l'exécutable nommé gatus.**

```
go build -o gatus .
```

Partie IV - Test Initial avec un Seul Service

Avant la configuration complète, effectuer un premier test avec un service simple (Google).

4.1 - Créer un fichier de configuration minimal

➢ **Action : Créer le fichier de configuration : nano config.yaml**

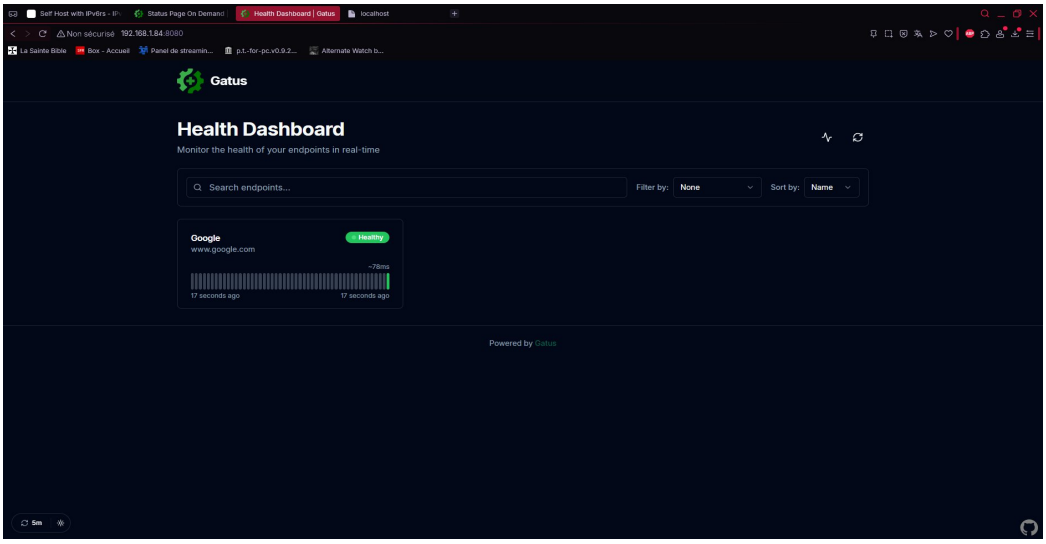
```
  GNU nano 7.2
endpoints:
  - name: Google
    url: "https://www.google.com"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 500"
```

```
endpoints:
- name: Google
  url: "https://www.google.com"
  interval: 1m
  conditions:
  - "[STATUS] == 200"
  - "[RESPONSE_TIME] < 500"
```

4.2 - Lancer Gatus et vérifier dans le navigateur

```
./gatus serve --config=config.yaml
```

```
http://192.168.1.84:8080
```



✓ **Résultat :** *Accéder à http://192.168.1.84:8080 — Gatus affiche un tableau de bord avec le service Google en vert (Healthy).*

Partie V - Configuration Complète de Gatus

Après le test, créer une configuration avancée avec plusieurs services, stockage SQLite et alertes Discord.

> ➢ **Action : Créer le fichier de configuration complet : nano /root/gatus/config/config.yaml**

```yaml
# ============================================================
# GATUS - Configuration
# ============================================================

# --------------------------
# STOCKAGE DES DONNÉES
# --------------------------
storage:
  type: sqlite
  path: data.db

# --------------------------
# ALERTING (DISCORD)
# --------------------------
alerting:
  discord:
    webhook-url: "https://discord.com/api/webhooks/1431034467715121274/ujVeMdKwHsg0RcGhYrGixhq9RpWQnHFjQj
    default-alert:
      description: "Un service surveillé par Gatus est en panne 🔴"
      send-on-resolved: true

# --------------------------
# ENDPOINTS - SERVICES À SURVEILLER
# --------------------------
endpoints:
  # ---- Production ----
  - name: Mon Site Web Principal
    group: Production
    url: "https://kalandy.carrd.co"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 500"
      - "[CERTIFICATE_EXPIRATION] > 72h"
    alerts:
      - type: discord
        failure-threshold: 3
        success-threshold: 2

  # ---- Local Network ----
  - name: Serveur Web Local
    group: Réseau Local
    url: "http://192.168.1.84:8080/health"
    interval: 30s
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 300"

  - name: NAS - ICMP Ping
    group: Réseau Local
    url: "icmp://192.168.1.20"
    interval: 1m
    conditions:
      - "[CONNECTED] == true"

  # ---- DNS Check ----
  - name: DNS Google
    group: Réseau
    url: "8.8.8.8"
    interval: 1m
    dns:
      query-name: "google.com"
      query-type: "A"
    conditions:
      - "[BODY] == pat(*.*.*.*)"
      - "[DNS_RCODE] == NOERROR"

  # ---- Test externe ----
  - name: API Github
    group: Externe
    url: "https://api.github.com"
    interval: 1m
    conditions:
      - "[STATUS] == 200"
      - "[RESPONSE_TIME] < 400"
```

Partie VI - Création du Service systemd

Un service systemd permet à Gatus de démarrer automatiquement, de s'exécuter en arrière-plan et de redémarrer automatiquement en cas de crash.

6.1 - Créer le fichier de service

➢ **Action : Créer le fichier de service : sudo nano /etc/systemd/system/gatus.service**

```
[Unit]
Description=Gatus monitoring service
After=network.target

[Service]
User=root
WorkingDirectory=/root/gatus
ExecStart=/root/gatus/gatus serve
Restart=always

[Install]
WantedBy=multi-user.target
```

```
[Unit]
Description=Gatus monitoring service
After=network.target

[Service]
User=root
WorkingDirectory=/root/gatus
ExecStart=/root/gatus/gatus serve
Restart=always

[Install]
WantedBy=multi-user.target
```

6.2 - Activer et démarrer le service

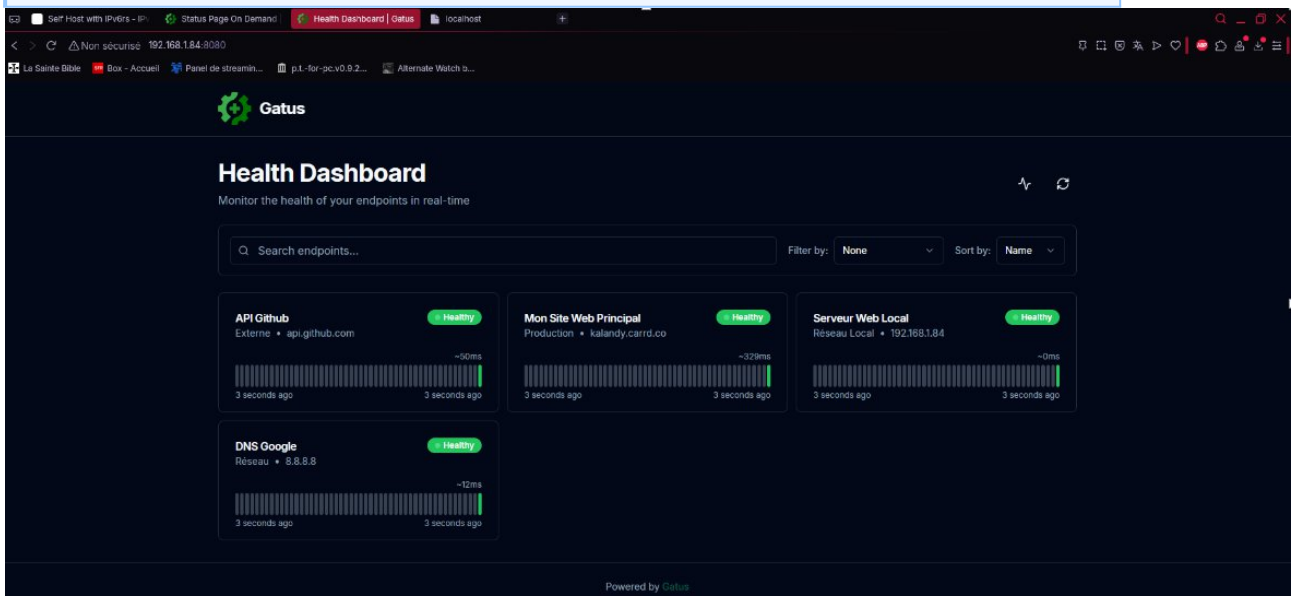
```
sudo systemctl daemon-reload
sudo systemctl enable gatus
sudo systemctl start gatus
sudo systemctl status gatus
```

```
● gatus.service - Gatus monitoring service
     Loaded: loaded (/etc/systemd/system/gatus.service; enabled; preset: enabled)
     Active: active (running) since Fri 2025-10-24 00:15:10 CEST; 20min ago
   Main PID: 6216 (gatus)
      Tasks: 6 (limit: 2315)
     Memory: 16.8M
        CPU: 662ms
     CGroup: /system.slice/gatus.service
             └─6216 /root/gatus/gatus serve
```

✓ **Résultat :** *Le service est actif et affiche l'état "running".*

Partie VII - Accès à l'Interface Web

➢ **Action : Accéder à l'interface via : http://192.168.1.84:8080**



✓ **Résultat :** *Le tableau de bord affiche tous les services surveillés, leur état et leur temps de réponse.*

Partie VIII - Test du Système d'Alerte

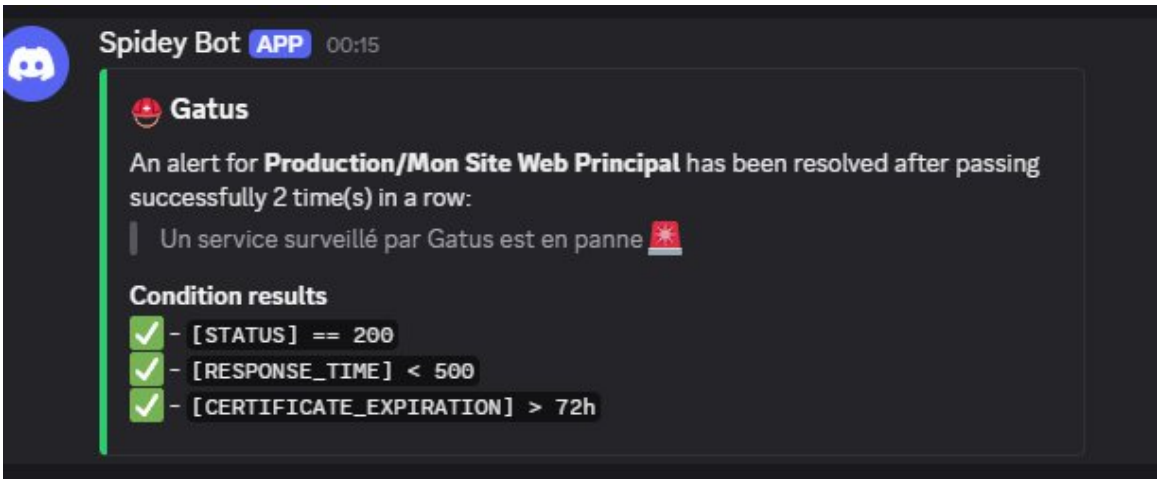
Pour valider le système d'alerte, provoquer volontairement une erreur en modifiant l'URL d'un service surveillé.

8.1 - Rendre un service inaccessible

➢ **Action : Modifier l'URL d'un service par un site inexistant dans config.yaml, puis redémarrer Gatus.**

```
# Dans config.yaml :
url: "https://fake.kalandy.carrd.co"

sudo systemctl restart gatus
```



✓ **Résultat :** *Après 3 échecs consécutifs, une alerte Discord est envoyée. Lorsque le service redevient accessible, une notification de rétablissement est automatiquement envoyée.*

Partie IX - Conclusion

Gatus est désormais entièrement configuré. Il permet de : superviser plusieurs services internes et externes, conserver l'historique des vérifications, envoyer des alertes via Discord, et redémarrer automatiquement grâce à systemd.

Mission Gatus complétée — Supervision réseau opérationnelle sur Debian 12.

— Fin de la documentation —